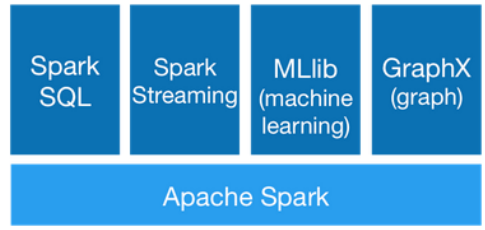
Big Data
(MEI)

João Pedro Costa
jcosta@isec.pt

1

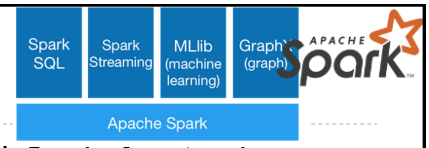
Spark

- Spark SQL
- PySpark



2

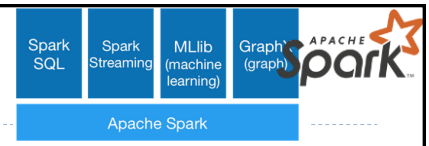
Spark SQL



- integrates relational processing with Sparks functional programming.
- Spark module for structured data processing
- blurs the line between RDD and relational table
- runs on top of the **Spark Core**.
- allows developers to access relational data from Hive tables and parquet files
- Runs unmodified Hive queries on current data
- rewrites the Hive frontend and meta store, allowing full compatibility with current Hive data, queries

3

Spark SQL



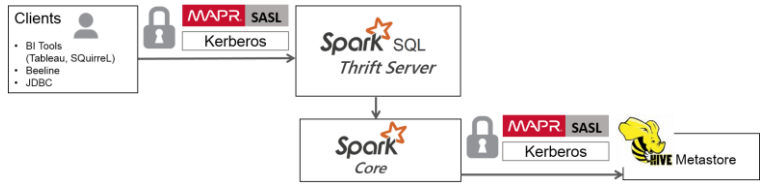
- module for structured data processing
- the interfaces provide Spark with more information about the structure of both the data and the computation being performed
- uses this extra information to perform extra optimizations
- When computing a result, the same execution engine is used, independent of which API/language you are using to express the computation
- allows developers switch between different APIs

4

Spark SQL

Spark SQL | Spark Streaming | MLlib (machine learning) | Graph (graph) | Apache Spark

- One use of Spark SQL is to execute SQL queries
- can read data from an existing Hive installation
- When running SQL from within another programming language the results will be returned as a Dataset/DataFrame.
- can interact using the command-line (spark-sql) or over JDBC/ODBC



5

Spark SQL

Spark SQL | Spark Streaming | MLlib (machine learning) | Graph (graph) | Apache Spark

Assume we have a small dataset of people:

```
Name     Age   City
Alice    25    Porto
Bob      30    Lisbon
Charlie  35    Porto
```

RDD

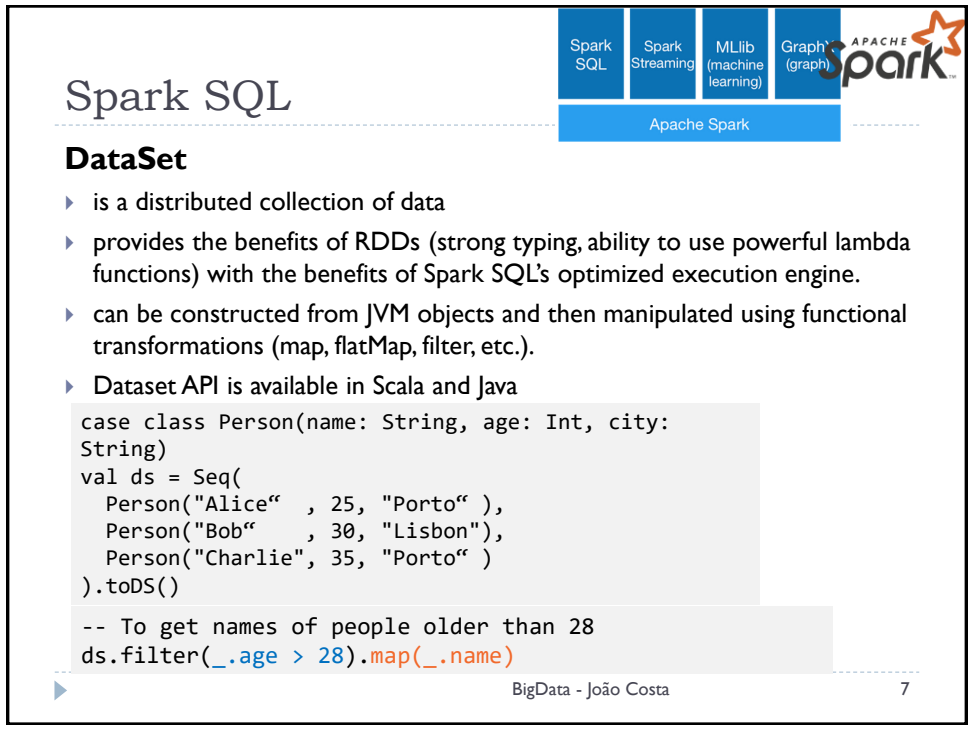
- stores data as **raw objects** (tuples, strings, etc.)

```
rdd = spark.sparkContext.parallelize(
[  ("Alice",   25, "Porto“ ),
   ("Bob",     30, "Lisbon"),
   ("Charlie", 35, "Porto“ )
])
```

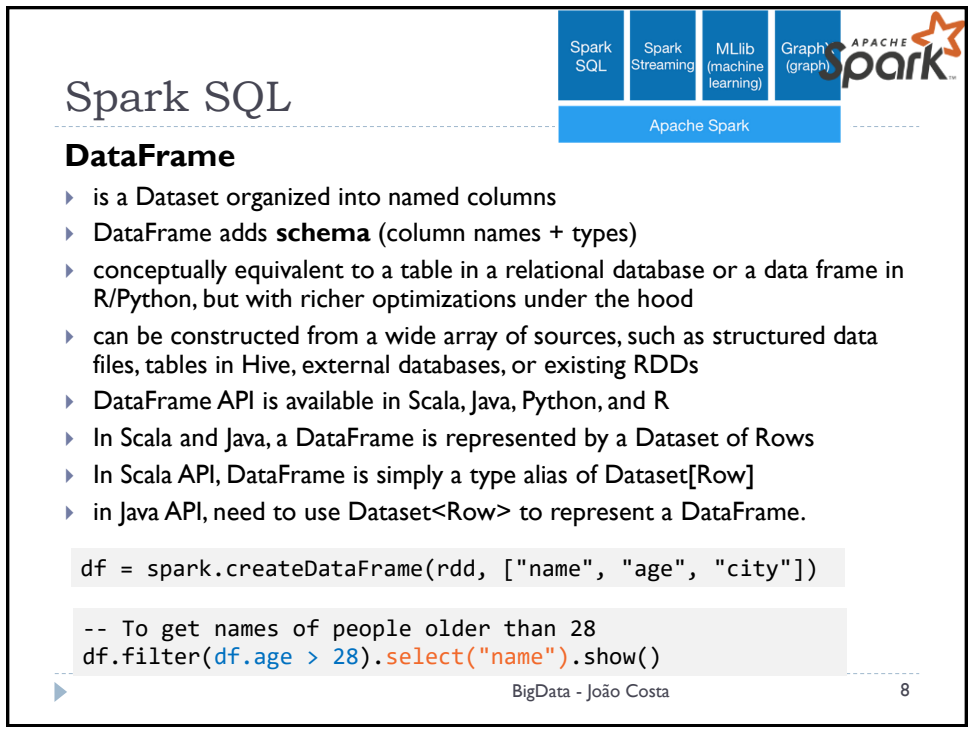
- No column names, no types enforced

```
-- To get names of people older than 28
rdd.filter(lambda x: x[1] > 28).map(lambda x: x[0]).collect()
```


6


Spark SQL
Spark Streaming
MLlib (machine learning)
Graph (graph)
Apache Spark
Spark SQL
DataSet
is a distributed collection of data
provides the benefits of RDDs (strong typing, ability to use powerful lambda functions) with the benefits of Spark SQL's optimized execution engine.
can be constructed from JVM objects and then manipulated using functional transformations (map, flatMap, filter, etc.).
Dataset API is available in Scala and Java
case class Person(name: String, age: Int, city: String)
val ds = Seq(
Person("Alice" , 25, "Porto"),
Person("Bob" , 30, "Lisbon"),
Person("Charlie", 35, "Porto")
).toDS()
-- To get names of people older than 28
ds.filter(_.age > 28).map(_.name)
BigData - João Costa
7

7


Spark SQL
Spark Streaming
MLlib (machine learning)
Graph (graph)
Apache Spark
Spark SQL
DataFrame
is a Dataset organized into named columns
DataFrame adds schema (column names + types)
conceptually equivalent to a table in a relational database or a data frame in R/Python, but with richer optimizations under the hood
can be constructed from a wide array of sources, such as structured data files, tables in Hive, external databases, or existing RDDs
DataFrame API is available in Scala, Java, Python, and R
In Scala and Java, a DataFrame is represented by a Dataset of Rows
In Scala API, DataFrame is simply a type alias of Dataset[Row]
in Java API, need to use Dataset<Row> to represent a DataFrame.
df = spark.createDataFrame(rdd, ["name", "age", "city"])
-- To get names of people older than 28
df.filter(df.age > 28).select("name").show()
BigData - João Costa
8

8

Spark SQL

	RDD	DataFrame	Dataset
Nível de Abstração	Baixo	Alto	Alto
Esquema	Sem esquema (coleção de objetos)	Organizado em colunas (com esquema)	Organizado em colunas (com esquema e tipos)
API	Funcional (map,filter,...)	Declarativa (SQL-like, select, filter, etc.)	Funcional e declarativa com tipagem estática
Otimização	Sem otimização interna	Catalyst Optimizer	Catalyst Optimizer
Segurança de Tipos	Não (dinâmico)	Não (dinâmico)	Sim (estático)
Uso Principal	Operações complexas, controle granular	Análise de dados estruturados e semi-estruturados	Análise de dados estruturados com tipagem estática
Linguagens Suportadas	Python,Java,Scala	Python, Java, Scala,R	Scala e Java

9

PySpark vs Scala

	Python	Scala
Performance	Slower	Faster (Spark in written in Scala)
Learning Curve	Simpler syntax Easy to learn	Complex syntax Harder to learn
Code Readability	Better Readability and maintenance	Complex Readability and maintenance
Data Science Library	Rich set of libraries for data visualization and model building	Lacks data visualization and model

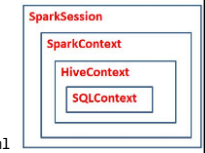
10

PySpark

the entry-point of a PySpark program is a SparkContext object

SparkContext

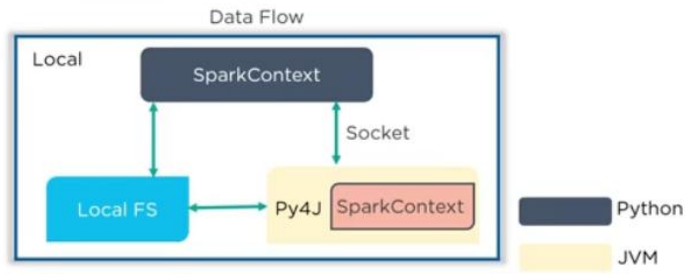
- allows you to connect to a Spark cluster
- allows you to create RDDs
- The `local[*]` string is a special string denoting that you're using a local cluster (running in single-machine mode)
  - The * tells Spark to create as many worker threads as logical cores on your machine



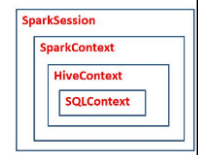
https://spark.apache.org/docs/3.5.1/api/python/reference/api/pyspark.SparkContext.html

PySpark - SparkContext

- The entry-point of any PySpark program
- Used by the driver process to establish communication with the cluster and resource managers to coordinate and execute jobs



- Enables access to SQLContext and HiveContext



PySpark - SparkContext

- represents the connection to a Spark cluster
- can be used to create RDD and broadcast variables on that cluster
- at least the master and app name should be set

```
pyspark.SparkContext(
    master      = None,
    appName     = None,
    sparkHome   = None,
    pyFiles     = None,
    environment= None,
    batchSize   = 0,
    conf        = None,
    gateway     = None
  )
```

```
from pyspark import SparkContext
sc = SparkContext( appName='app1',
        master='spark://10.84.128.47:7077')
```

```
Setting default log level to "WARN". ●●●
```

```
sc
```

SparkContext

Spark UI

Version	v3.5.1
Master	spark://10.84.128.47:7077
AppName	app1

PySpark - SparkContext

- represents the connection to a Spark cluster
- can be used to create RDD and broadcast variables on that cluster
- at least the master and app name should be set

```
pyspark.SparkContext(
    master      = None,
    appName     = None,
    sparkHome   = None,
    pyFiles     = None,
    environment= None,
    batchSize   = 0,
    conf        = None,
    gateway     = None
  )
```

```
from pyspark import SparkContext, SparkConf
conf = SparkConf() \
    .setAppName('app') \
    .setMaster("spark://10.84.128.47:7077")
sc = SparkContext( conf = conf)
sc
```

SparkContext

Spark UI

Version	v3.5.1
Master	spark://10.84.128.47:7077
AppName	app

PySpark - SparkContext

master *string*
Cluster URL to connect to (e.g. spark://host:port, local[4])
appName *string*
A name for the job, to display on the cluster web UI
sparkHome *string*
Location where Spark is installed on cluster nodes
pyFiles *list*
Collection of .zip or .py files to send to the cluster and add to PYTHONPATH. paths on the local file system or HDFS,HTTP or FTP URLs
environment *dict*
dictionary of environment variables to set on worker nodes
batchSize *int*
number of Python objects represented as a single Java object.
Set 1 to disable batching, 0 to automatically choose the batch size based on object sizes, or -1 to use an unlimited batch size
conf *pyspark.SparkConf*
An object setting Spark properties

PySpark - SparkConf

- Provides configurations to run a Spark application

setAppName(*value* **)**
to name of the application

setMaster(*value* **)**
to set the master URL

set(*key* **,** *value* **)**
to set a configuration value of a key

get(*key, defaultValue=None* **)**
to get a configuration value of a key

getAll()
to get all configuration key, values

```
from pyspark import SparkConf
conf = SparkConf() \
    .setAppName('App01') \
    .setMaster("spark://10.84.129.50:7077")
conf.getAll()
```

```
[('spark.app.name', 'App01'), ('spark.master',
'spark://10.84.129.50:7077')]
```

```
conf.get("spark.master")
```

```
'spark://10.84.129.50:7077'
```

```
conf.set("ui.enable","false")
```

```
<pyspark.conf.SparkConf at 0x7ff834626770>
```

```
conf.getAll()
```

```
[('spark.app.name', 'App01'),
 ('spark.master', 'spark://10.84.129.50:7077'),
 ('ui.enable', 'false')]
```

PySpark - SparkConf

- configuration for a Spark application
- Used to set various Spark parameters as key-value pairs
- All setter methods in this class support chaining

```
from pyspark import SparkConf
conf = SparkConf().setAppName('app').setMaster("spark://10.84.128.47:7077")
```

```
from pyspark import SparkConf
conf = SparkConf() \
    .setAppName('app') \
    .setMaster("spark://10.84.128.47:7077")
```

```
type(conf)
```

```
pyspark.conf.SparkConf
```

```
conf.get('spark.master')
```

```
'spark://10.84.128.47:7077'
```


17

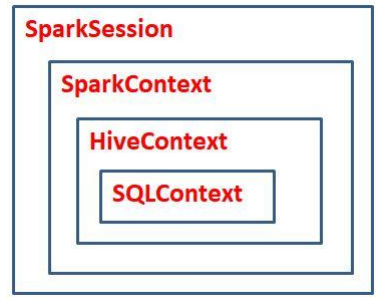
PySpark - SparkConf

contains(key)
Does this configuration contain a given key?
get(key[, defaultValue])
Get the configured value for some key, or return a default otherwise
getAll()
Get all values as a list of key-value pairs.
set(key, value)
Set a configuration property.
setAll(pairs)
Set multiple parameters, passed as a list of key-value pairs.
setAppName(value)
Set application name.
setExecutorEnv([key, value, pairs])
Set an environment variable to be passed to executors.
setIfMissing(key, value)
Set a configuration property, if not already set.
setMaster(value)
Set master URL to connect to.
setSparkHome(value)
Set path where Spark is installed on worker nodes.
toDebugString()
Returns a list of key=value pairs, one per line

18

PySpark

- SparkSession
- SparkContext
- SQLContext
- HiveContext



the entry-point of a PySpark program is a SparkContext object

PySpark - SparkContext

SparkFiles

- Allows the upload of files with **`sc.addFile(...)`**

```
from pyspark import SparkContext, SparkConf
conf = SparkConf() \
    .setAppName('app') \
    .setMaster("spark://10.84.128.47:7077")
sc = SparkContext( conf = conf)
sc
```

```
sc.addFile('t.py')
```

```
from pyspark import SparkFiles
SparkFiles.getRootDirectory()
```

```
'/tmp/spark-7b679d90-7d33-468b-8aab-c3f01bdc3ae6/userFiles-0
```

A worker can get it with **`SparkFiles.get(...)`**

```
f = SparkFiles.get('t.py')
print(f)
```

```
/tmp/spark-7b679d90-7d33-468b-8aab-c3f01bdc3ae6/userFiles-0a731c19-cf8f-4045-8093-bea17953b616/t.py
```

PySpark – SparkContext file methods

textFile(name[, minPartitions, use_unicode])

Read a text file from HDFS, a local file system (available on all nodes), or any Hadoop-supported file system URI, and return it as an RDD of Strings

wholeTextFiles(path[, minPartitions, …])

Read a directory of text files from HDFS, a local file system (available on all nodes), or any Hadoop-supported file system URI.

binaryFiles(path[, minPartitions])

Read a directory of binary files from HDFS, a local file system (available on all nodes), or any Hadoop-supported file system URI as a byte array.

binaryRecords(path, recordLength)

Load data from a flat binary file, assuming each record is a set of numbers with the specified numerical format, and the number of bytes per record is constant

hadoopFile(path, inputFormatClass, keyClass, …)

Read an 'old' Hadoop InputFormat with arbitrary key and value class from HDFS, a local file system (available on all nodes), or any Hadoop-supported file system URI

sequenceFile(path[, keyClass, valueClass, …])

Read a Hadoop SequenceFile with arbitrary key and value Writable class from HDFS, a local file system (available on all nodes), or any Hadoop-supported file system URI

PySpark – SparkContext textFile

```
>>> import os
>>> import tempfile
>>> with tempfile.TemporaryDirectory() as d:
...     path1 = os.path.join(d, "text1")
...     path2 = os.path.join(d, "text2")
...
...     # Write a temporary text file
...     sc.parallelize(["x", "y", "z"]).saveAsTextFile(path1)
...
...     # Write another temporary text file
...     sc.parallelize(["aa", "bb", "cc"]).saveAsTextFile(path2)
...
...     # Load text file
...     collected1 = sorted(sc.textFile(path1, 3).collect())
...     collected2 = sorted(sc.textFile(path2, 4).collect())
...
...     # Load two text files together
...     collected3 = sorted(sc.textFile('{},{}'.format(path1, path2), 5).collect())
```

```
>>> collected1
['x', 'y', 'z']
>>> collected2
['aa', 'bb', 'cc']
>>> collected3
['aa', 'bb', 'cc', 'x', 'y', 'z']
```

PySpark – SparkContext wholeTextFiles

```
>>> import os
>>> import tempfile
>>> with tempfile.TemporaryDirectory() as d:
...     # Write a temporary text file
...     with open(os.path.join(d, "1.txt"), "w") as f:
...         _ = f.write("123")
...
...     # Write another temporary text file
...     with open(os.path.join(d, "2.txt"), "w") as f:
...         _ = f.write("xyz")
...
...     collected = sorted(sc.wholeTextFiles(d).collect())
>>> collected
[('.../1.txt', '123'), ('.../2.txt', 'xyz')]
```


PySpark – SparkContext file methods

hadoopFile(path, inputFormatClass, keyClass, …)

Read an 'old' Hadoop InputFormat with arbitrary key and value class from HDFS, a local file system (available on all nodes), or any Hadoop-supported file system URI

hadoopRDD(inputFormatClass, keyClass, valueClass)

Read an 'old' Hadoop InputFormat with arbitrary key and value class, from an arbitrary Hadoop configuration, which is passed in as a Python dict.

newAPIHadoopFile(path, inputFormatClass, …)

Read a 'new API' Hadoop InputFormat with arbitrary key and value class from HDFS, a local file system (available on all nodes), or any Hadoop-supported file system URI

newAPIHadoopRDD(inputFormatClass, keyClass, …)

Read a 'new API' Hadoop InputFormat with arbitrary key and value class, from an arbitrary Hadoop configuration, which is passed in as a Python dict.

PySpark – SparkContext RDD methods

parallelize(c [, numSlices])
Distribute a local Python collection to form an RDD.

range(start [, end, step, numSlices])
Create a new RDD of int containing elements from start to end (exclusive), increased by step every element.

emptyRDD()
Create an RDD that has no partitions or elements.

union(rdds)
*Build the union of a list of RDDs.**

PySpark – SparkContext methods

accumulator(value [, accum_param])
Create an Accumulator with the initial value, using a given AccumulatorParam helper object to define how to add values of the data type if provided.

broadcast(value)
Broadcast a read-only variable to the cluster, returning a Broadcast object for reading it in distributed functions.

pickleFile(name [, minPartitions])
Load an RDD previously saved using RDD.saveAsPickleFile() method.

PySpark – SparkContext methods

cancelAllJobs()
Cancel all jobs that have been scheduled or are running.
cancelJobGroup(groupId)
Cancel active jobs for the specified group.
getCheckpointDir()
Return the directory where RDDs are checkpointed.
getOrCreate([conf])
Get or instantiate a SparkContext and register it as a singleton object.
runJob(rdd, partitionFunc[, partitions, …])
Executes the given partitionFunc on the specified set of partitions, returning the result as an array of elements.
setCheckpointDir(dirName)
Set the directory under which RDDs are going to be checkpointed.
setJobDescription(value)
Set a human readable description of the current job.
setJobGroup(groupId, description[, …])
Assigns a group ID to all the jobs started by this thread until the group ID is set to a different value or cleared.
setLocalProperty(key, value)
Set a local property that affects jobs submitted from this thread, such as the Spark fair scheduler pool.
setLogLevel(logLevel)
Control our logLevel.
setSystemProperty(key, value)
Set a Java system property, such as spark.executor.memory
sparkUser()
Get SPARK_USER for user who is running SparkContext.
statusTracker()
Return StatusTracker object
stop()
Shut down the SparkContext.

PySpark – SparkContext methods

getCheckpointDir()
Return the directory where RDDs are checkpointed.
setCheckpointDir(dirName)
Set the directory under which RDDs are going to be checkpointed.
setJobDescription(value)
Set a human readable description of the current job.
setJobGroup(groupId, description[, …])
Assigns a group ID to all the jobs started by this thread until the group ID is set to a different value or cleared.
setLocalProperty(key, value)
Set a local property that affects jobs submitted from this thread, such as the Spark fair scheduler pool.
setLogLevel(logLevel)
Control our logLevel.
setSystemProperty(key, value)
Set a Java system property, such as spark.executor.memory

PySpark – SparkContext methods

```
cancelAllJobs()
Cancel all jobs that have been scheduled or are running.
cancelJobGroup(groupId)
Cancel active jobs for the specified group.
getCheckpointDir()
Return the directory where RDDs are checkpointed.
getOrCreate([conf])
Get or instantiate a SparkContext and register it as a singleton object.
runJob(rdd, partitionFunc[, partitions, …])
Executes the given partitionFunc on the specified set of partitions, returning the result as an array of elements.
setCheckpointDir(dirName)
Set the directory under which RDDs are going to be checkpointed.
setJobDescription(value)
Set a human readable description of the current job.
setJobGroup(groupId, description[, …])
Assigns a group ID to all the jobs started by this thread until the group ID is set to a different value or cleared.
setLocalProperty(key, value)
Set a local property that affects jobs submitted from this thread, such as the Spark fair scheduler pool.
setLogLevel(logLevel)
Control our logLevel.
setSystemProperty(key, value)
Set a Java system property, such as spark.executor.memory
sparkUser()
Get SPARK_USER for user who is running SparkContext.
statusTracker()
Return StatusTracker object
stop()
Shut down the SparkContext.
```


PySpark – SQLContext

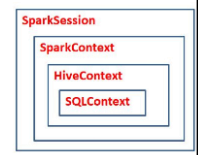
- The entry-point to SparkSQL module for structure data processing
- Can be used to sql-like operations over datasets and dataframes
- to create a SQLContext, first need to instantiate a SparkContext

```
from pyspark import SparkContext, SparkConf
from pyspark.sql import SQLContext

local = SparkConf().setAppName('app').setMaster("local[*]")
sc = SparkContext(conf=local)

sqlContext = SQLContext(sc)
sqlContext
```

```
<pyspark.sql.context.SQLContext at 0x7ff2969abd50>
```



Deprecated : use SparkSession

PySpark – HiveContext

- The entry-point to HiveContext and Hive Metadata

```
from pyspark import SparkContext, SparkConf
from pyspark.sql import HiveContext

local = SparkConf().setAppName('app').setMaster("local[*]")
sc = SparkContext(conf=local)

hiveContext = HiveContext(sc)
hiveContext
```

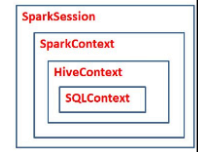
```
rs = hiveContext.sql("show tables");
rs.show()
```

```
+---------+---------+-----------+
|namespace|tableName|isTemporary|
+---------+---------+-----------+
+---------+---------+-----------+
```

```
hiveContext.tableNames()
```

```
[]
```

```
sc.stop()
```



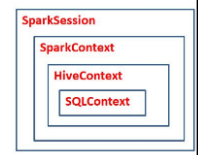
PySpark – SparkSession

- Replaces SQLContext and HiveContext
- Gives immediate access to SparkContext

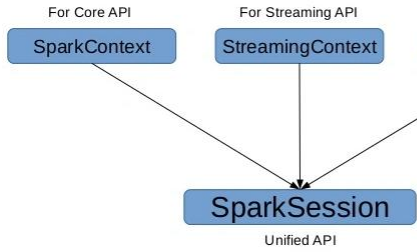
```
from pyspark.sql import SparkSession
spark = SparkSession.builder \
  .master("local[*]") \
  .appName("SparkHive") \
  .enableHiveSupport() \
  .getOrCreate()
```

```
df = spark.sql ("select * from movielens.u_genre order by genre desc")
print(df)
df.show(4)
```

```
DataFrame[genre: string, genreid: int]
+--------+-------+
|   genre|genreid|
+--------+-------+
| unknown|      0|
| Western|     18|
|     War|     17|
|Thriller|     16|
+--------+-------+
only showing top 4 rows
```



PySpark – SparkSession




```
# Two ways you can access spark context from spark session
sparkContext = spark._sc
# or
spark_context = spark.sparkContext
sparkContext
```

SparkContext

Spark UI

Version	v3.4.0
Master	local[*]
AppName	SparkHive

```
# To get SparkConf
sparkConf = sparkContext.getConf()
sparkConf.getAll()
```

```
[('spark.eventLog.enabled', 'true'),
 ('spark.executor.instances', '4'),
 ('spark.dynamicAllocation.maxExecutors', '4'),
```


PySpark – SparkSession

SparkSession Builder

- getOrCreate() - try to get a spark session or create a new one
- enableHiveSupport() – enables access to Hive metastore
- Gives immediate access to SparkContext

```
from pyspark.sql import SparkSession
```

```
spark = SparkSession \
  .builder \
  .master("local[*]") \
  .appName("SparkHive") \
  .enableHiveSupport() \
  .getOrCreate()
```

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName('bigdata')\
        .master("spark://10.84.128.47:7077") \
        .config("spark.driver.host","10.84.129.48") \
        .config("spark.executor.memory", "1g") \
        .config("spark.executor.cores", "2") \
        .config("spark.executor.instances", "2") \
        .getOrCreate()
spark
```

PySpark – HiveSupport

Hive – using HQL

```
from pyspark.sql import SparkSession
```

```
spark = SparkSession \
  .builder \
  .master("local[*]") \
  .appName("SparkHive") \
  .enableHiveSupport() \
  .getOrCreate()
```

```
# execute a CREATE TABLE sql statement
spark.sql("CREATE TABLE IF NOT EXISTS src(key INT, value STRING)")
```

```
# define Hive Table format
spark.sql("CREATE TABLE src1(id int) stored as parquet")
```

```
# spark is an existing SparkSession
spark.sql("LOAD DATA INPATH '/examples/05-spark/resources/kv1.txt' INTO TABLE src")
```


36

PySpark – HiveSupport

Hive – using HQL

```
from pyspark.sql import SparkSession
```

```
spark = SparkSession \
  .builder \
  .master("local[*]") \
  .appName("SparkHive") \
  .enableHiveSupport() \
  .getOrCreate()
```

```
# Queries are expressed in HiveQL
df = spark.sql("SELECT * FROM src")
df.show(5)
```

```
+---+-------+
|key|  value|
+---+-------+
|238|val_238|
| 86| val_86|
|311|val_311|
| 27| val_27|
|165|val_165|
+---+-------+
only showing top 5 rows
```

```
# Queries are expressed in HiveQL
df = spark.sql("SELECT key,value FROM src where key<10 order by key")
df.show(5)
```

```
+---+-----+
|key|value|
+---+-----+
|  0|val_0|
|  0|val_0|
|  0|val_0|
|  2|val_2|
|  4|val_4|
+---+-----+
only showing top 5 rows
```


37

PySpark

- The entry-point of any PySpark program is a SparkContext object.

```
import pyspark
sc = pyspark.SparkContext('local[*]')

txt = sc.textFile('file:////usr/share/doc/python3/copyright')
print(txt.count())
```

```
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
[Stage 0:>                                                          (0 + 2) / 2]
319
```

```
python_lines = txt.filter(lambda line: 'python' in line.lower())
print(python_lines.count())
```

```
52
```

```
sc.stop()
```


38

PySpark

- Connect to jdbc databases

```
jdbcDF = spark.read \
    .format("jdbc") \
    .option("url", "jdbc:postgresql:dbserver") \
    .option("dbtable", "schema.tablename") \
    .option("user", "username") \
    .option("password", "password") \
    .load()
```

```
jdbcDF2 = spark.read \
  .jdbc("jdbc:postgresql:dbserver", "schema.tablename",
        properties={"user":"username","password":"password"})
```


39

PySpark

- Connect to jdbc databases

```
jdbcDF3 = spark.read \
    .format("jdbc") \
    .option("url", "jdbc:postgresql://10.84.128.2:25432/dw") \
    .option("dbtable", "produtos") \
    .option("user", "f2ex2promo") \
    .option("password", "dw") \
    .load()
```


40

PySpark

- Connect to jdbc databases
- Saving data to a JDBC source

```
jdbcDF.write \
    .format("jdbc") \
    .option("url", "jdbc:postgresql:dbserver") \
    .option("dbtable", "schema.tablename") \
    .option("user", "username") \
    .option("password", "password") \
    .save()
```

```
jdbcDF2 = spark.write \
  .jdbc("jdbc:postgresql:dbserver", "schema.tablename",
        properties={"user":"username","password":"password"})
```


41

PySpark.sql

- Load data from file
  - `spark.read.load( file )`
  - `spark.read.load( file , format = format)`
  - `spark.read.load( file , format = format)`
  - `spark.read.format( format ).load( file)`

  - `Format :`
    - `text`
    - `csv`
    - `json`
    - `parquet`
    - `avro`
    - `orc`

PySpark

- Load data from file

```
df = spark.read.load("examples/src/main/resources/people.json",
          format="json")
```

```
df = spark.read.load("examples/src/main/resources/users.parquet",
          format="parquet")
```

PySpark

- Load JSON data from a HDFS file

```
from pyspark.sql import SparkSession
spark = SparkSession.builder \
   .getOrCreate()
```

```
df1 = spark.read.load("/examples/05-spark/resources/people.json", format="json")
```

```
df2 = spark.read.format("json")  .load("/examples/05-spark/resources/people.json")
```

```
df3 = spark.read.json("/examples/05-spark/resources/people.json")
```

```
df1.show(), df2.show(), df3.show()
```


PySpark

- Read json data into a DataFrame

```
[3]: from pyspark.sql import SparkSession

spark = SparkSession.builder\
    .appName("Python Spark SQL get started") \
    .getOrCreate()
df = spark.read \
     .json("file:///bigdata/spark/examples/src/main/resources/people.json")
df.show()
```

```
+----+-------+
| age|   name|
+----+-------+
|null|Michael|
|  30|   Andy|
|  19| Justin|
+----+-------+
```

PySpark

- Load data from CSV file

```
from pyspark.sql import SparkSession
spark = SparkSession.builder \
    .getOrCreate()
```

```
df = spark.read.format("csv") \
  .option("sep", ";") \
  .option("inferSchema", "true") \
  .option("header", "true") \
  .load("/examples/05-spark/resources/people.csv")
```

```
df.show()
```

```
+-----+---+---------+
| name|age|      job|
+-----+---+---------+
|Jorge| 30|Developer|
|  Bob| 32|Developer|
+-----+---+---------+
```

```
df.printSchema()
```

```
root
 |-- name: string (nullable = true)
 |-- age: integer (nullable = true)
 |-- job: string (nullable = true)
```


PySpark

- Load data from file, specific formats
  - `spark.read.load( file )`
  - **`spark.read.json( file )`**
  - **`spark.read.parquet( file )`**
  - **`spark.read.orc( file )`**

PySpark

Load a Parquet file

- Parquet files are self-describing so the schema is preserved
- a loaded parquet file is a DataFrame

```
from pyspark.sql import SparkSession
spark = SparkSession.builder \
   .getOrCreate()
```

```
df_parquet = spark.read \
   .parquet("/examples/05-spark/resources/users.parquet")
```

```
df_parquet.show()
```

```
[Stage 3:>
+------+--------------+----------------+
|  name|favorite_color|favorite_numbers|
+------+--------------+----------------+
|Alyssa|          null|  [3, 9, 15, 20]|
|   Ben|           red|              []|
+------+--------------+----------------+
```

```
df_parquet.printSchema()
```

```
root
 |-- name: string (nullable = true)
 |-- favorite_color: string (nullable = true)
 |-- favorite_numbers: array (nullable = true)
 |    |-- element: integer (containsNull = true)
```


48

PySpark

Load a Orc file

- Orc files are self-describing so the schema is preserved
- a loaded ORC file is a DataFrame

```
from pyspark.sql import SparkSession
spark = SparkSession.builder \
   .getOrCreate()
```

```
df_orc = spark.read\
   .orc("/examples/05-spark/resources/users.orc")
```

```
df_orc.show()
```

```
+------+--------------+----------------+
|  name|favorite_color|favorite_numbers|
+------+--------------+----------------+
|Alyssa|          null|  [3, 9, 15, 20]|
|   Ben|           red|              []|
+------+--------------+----------------+
```

```
df_orc.printSchema()
```

```
root
 |-- name: string (nullable = true)
 |-- favorite_color: string (nullable = true)
 |-- favorite_numbers: array (nullable = true)
 |    |-- element: integer (containsNull = true)
```


49

PySpark

- Load and save DF to file

```
df_name= df_parquet.select("name")
df_name.show()
```

```
[Stage 3:>
+------+
|  name|
+------+
|Alyssa|
|   Ben|
+------+
```

```
df_name.write.parquet("/examples/05-spark/resources/names.parquet")
```

```
df_name.write.orc("/examples/05-spark/resources/names.orc")
```

```
df_name.write.csv("/examples/05-spark/resources/names.csv")
```


PySpark

Load Hive Table

```
from pyspark.sql import SparkSession
spark = SparkSession.builder \
  .master("local[*]") \
  .appName("SparkHive") \
  .enableHiveSupport() \
  .getOrCreate()
spark._sc.setLogLevel("ERROR")
```

```
spark.sql("USE DATABASE movielens")
```

```
# execute a SELECT sql statement
df1 = spark.sql("SELECT * \
                 FROM movielens.u_user u \
                 order by age ")
```

```
df1.printSchema()
```

```
root
 |-- userid: integer (nullable = true)
 |-- age: integer (nullable = true)
 |-- gender: string (nullable = true)
 |-- ocupation: string (nullable = true)
 |-- zip: integer (nullable = true)
```

PySpark

- DataFrame

```
# show schema
f.printSchema()
```

```
# Select ...
df.select("name")
```

```
# execute a SELECT sql statement
df1 = spark.sql("SELECT * \
                 FROM movielens.u_user u \
                 order by age ")
```

```
df1.printSchema()
```

```
root
 |-- userid: integer (nullable = true)
 |-- age: integer (nullable = true)
 |-- gender: string (nullable = true)
 |-- ocupation: string (nullable = true)
 |-- zip: integer (nullable = true)
```

```
# show distinct ocupations
#df2 = spark.sql("SELECT distinct ocupation FROM movielens.u_user")
df2 = df1.select("ocupation").distinct()
df2.show(4)
```

```
+---------+
|ocupation|
+---------+
|librarian|
|  retired|
|   lawyer|
|     none|
+---------+
only showing top 4 rows
```


PySpark

- DataFrame - Select

```
# Select everybody, but increment the age by 1
df1.select(df1['ocupation'], df1['age'] + 1).show()
```

```
# Select everybody, but increment the age by 1
df3 = df1.select(df1['ocupation'], df1['age'] + 1)
df3.show(4)
```

```
+---------+---------+
|ocupation|(age + 1)|
+---------+---------+
|  student|        8|
|  student|       11|
|     none|       12|
|  student|       14|
+---------+---------+
only showing top 4 rows
```

PySpark

- DataFrame - Filter

```
# Select people older than 21
df.filter(df['age'] > 21).show()
```

```
# Filter, select people older than 21
df4 = df1.filter(df1['age'] > 21)
df4.show(4)
```

```
+------+---+------+-------------+-----+
|userid|age|gender|    ocupation|  zip|
+------+---+------+-------------+-----+
|   180| 22|     F|administrator|60202|
|   282| 22|     M|administrator|20057|
|   216| 22|     M|     engineer| 2215|
|   217| 22|     M|        other|11727|
+------+---+------+-------------+-----+
only showing top 4 rows
```


PySpark

- DataFrame - GroupBy

```
# Count people by age
df.groupBy("age").count().show()
```

```
# Count people by age
df5 = df1.groupBy('age').count()
df5.show(4)
```

```
+---+-----+
|age|count|
+---+-----+
| 31|   25|
| 65|    3|
| 53|   12|
| 34|   17|
+---+-----+
only showing top 4 rows
```

PySpark

DataFrame - temporary view

- session-scoped view
- that disappear when the session ends

```
df
```

```
DataFrame[name: string, age: int, job: string]
```

```
df.printSchema()
```

```
root
 |-- name: string (nullable = true)
 |-- age: integer (nullable = true)
 |-- job: string (nullable = true)
```

```
# Register the DataFrame as a SQL temporary view
df.createOrReplaceTempView("people")
```

```
sqlDF = spark.sql("SELECT name, age FROM people where age > 30")
sqlDF.show()
```

```
+----+---+
|name|age|
+----+---+
| Bob| 32|
+----+---+
```


PySpark

- DataFrame - Global Temporary view
  - temporary view shared among all sessions
  - and keep alive until the Spark application terminates
  - in database global_temp,
  - use the qualified name to refer it,
    - e.g. `SELECT * FROM global_temp.view1`

```
# Register the DataFrame as a SQL global view
# shared among all sessions
df.createOrReplaceGlobalTempView("people")
```

```
sqlDF = spark.sql("SELECT name, age FROM global_temp.people")
sqlDF.show()
```

```
[Stage 12:>
+-----+---+
| name|age|
+-----+---+
|Jorge| 30|
|  Bob| 32|
+-----+---+
```

PySpark

- Querying

```
from pyspark.sql import SparkSession
spark = SparkSession.builder \
  .master("local[*]") \
  .appName("SparkHive") \
  .enableHiveSupport() \
  .getOrCreate()
spark._sc.setLogLevel("ERROR")
```

```
spark.sql("USE DATABASE movielens")
```

```
# execute a CREATE TABLE sql statement
df = spark.sql("SELECT gender, count(*) \
                FROM movielens.u_data d join movielens.u_user u \
                where d.userid=u.userid \
                group by gender")
df.show()
```

```
+------+--------+
|gender|count(1)|
+------+--------+
|     F|   51480|
|     M|  148520|
+------+--------+
```


58

Load Text into RDD

```
from pyspark.sql import SparkSession
from pyspark.sql import Row
spark = SparkSession.builder \
   .master("local[*]") \
   .getOrCreate()
sc = spark.sparkContext
```

```
# Load a text file and convert each line to a Row.
lines = sc.textFile("/examples/05-spark/resources/people.txt")
```

```
/examples/05-spark/resources/people.txt MapPartitionsRDD[1] at textFile at Nativ
```

```
parts = lines.map(lambda l: l.split(","))
```

```
PythonRDD[2] at RDD at PythonRDD.scala:53
```

```
people = parts.map(lambda p: Row(name=p[0], age=int(p[1])))
```

```
PythonRDD[3] at RDD at PythonRDD.scala:53
```

```
# Infer the schema, and register the DataFrame as a table.
schemaPeople = spark.createDataFrame(people)
schemaPeople.createOrReplaceTempView("people")
```

```
# SQL can be run over DataFrames that have been registered as a table.
teenagers = spark.sql("SELECT name FROM people WHERE age >= 13 AND age <= 19")
```

```
DataFrame[name: string]
```

```
# The results of SQL queries are Dataframe objects.
# rdd returns the content as an :class:`pyspark.RDD` of :class:`Row`.
teenNames = teenagers.rdd.map(lambda p: "Name: " + p.name).collect()
for name in teenNames: print(name)
```

```
[Stage 1:>                                                          (0 + 2) / 2]
Name: Justin
```


59